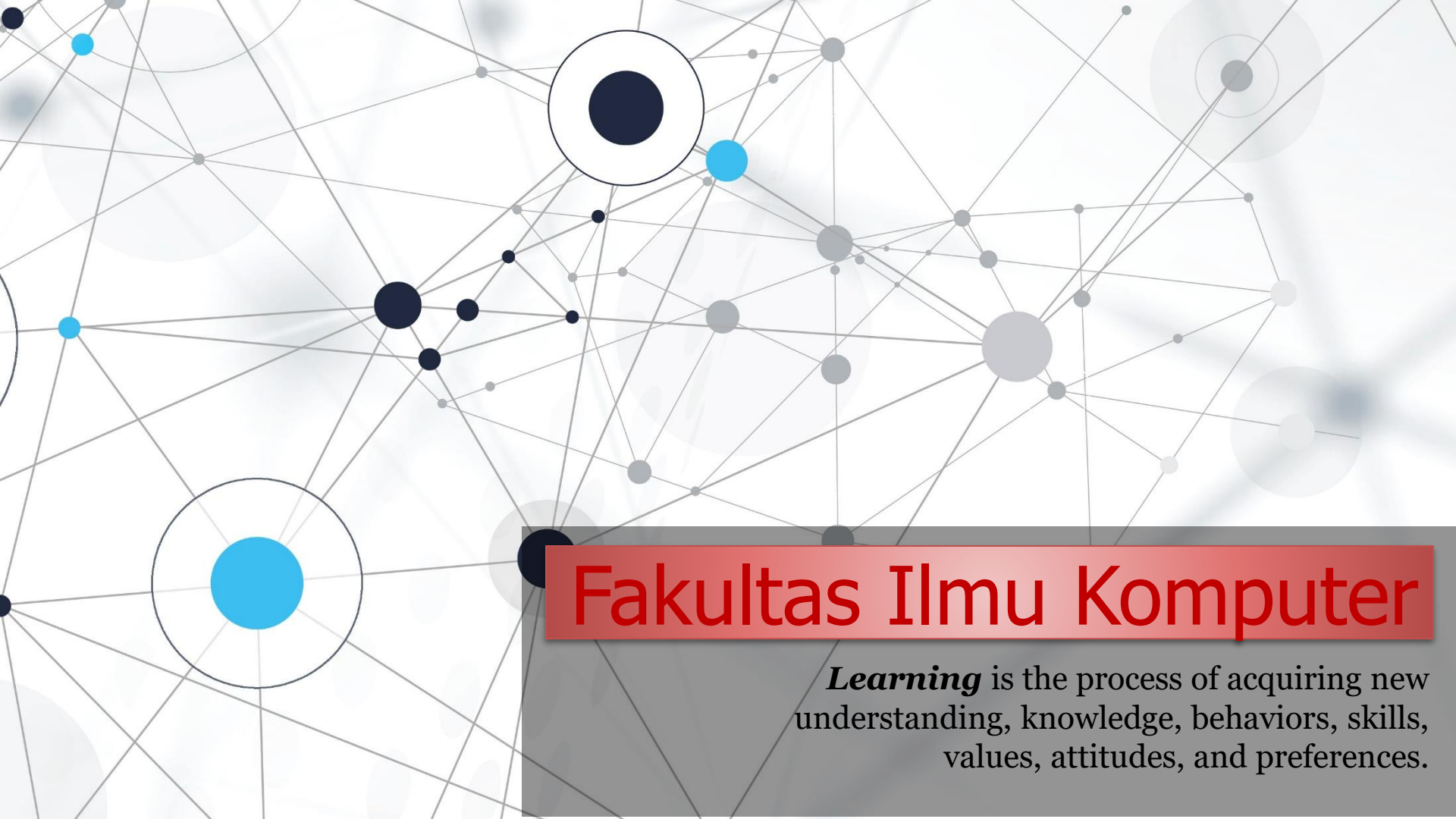


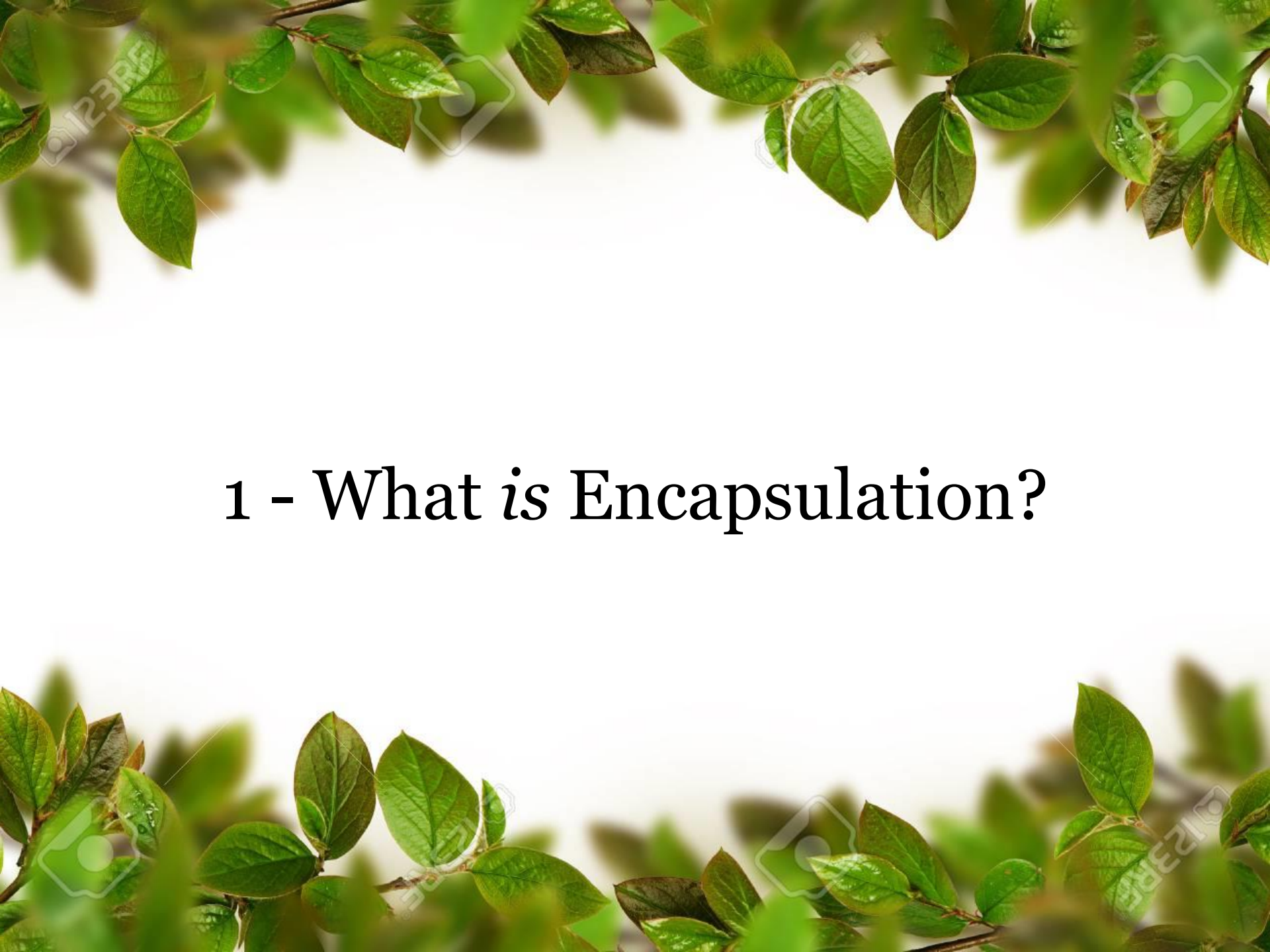
An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Setters & Getters



The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub. The text is centered in the middle of the slide.

1 - What is Encapsulation?

What is Encapsulation?



Encapsulation is a programming concept used to organize data or properties in an object, so that **data/variables can only be accessed through methods** or functions that provided by the object.

In this way, the **data and even method can be better organized and securely controlled**, thus **avoiding unwanted changes or unauthorized access from outside** the object.

Contoh Pemrograman Java

```
public class Mobil {  
    // atribut public  
    public String nama;  
    public String warna;  
  
    // atribut private  
    private int tahunPembuatan;  
    private int kecepatan;  
  
    // method public untuk mengubah kecepatan  
    public void ubahKecepatan(int kecepatanBaru) {  
        this.kecepatan = kecepatanBaru;  
    }  
  
    // method private untuk mematikan mesin  
    private void matikanMesin() {  
        this.kecepatan = 0;  
    }  
}
```

Atribut **tahunPembuatan** dan metode **matikanMesin** diatur sebagai *private access*, sehingga hanya dapat diakses dari dalam class Mobil.

Atribut dan metode tersebut tidak dapat diakses langsung dari luar objek Mobil, melainkan hanya dapat diakses melalui **metode atau fungsi publik** yang disediakan oleh objek Mobil, seperti metode **ubahKecepatan**.

Contoh Pemrograman Java

```
class User {  
    private String username;  
    private String password;  
  
    // ini method setter  
    public void setUsername(String username){  
        this.username = username;  
    }  
  
    public void setPassword(String password){  
        this.password = password;  
    }  
  
    // ini method getter  
    public String getUsername(){  
        return this.username;  
    }  
  
    public String getPassword(){  
        return this.password;  
    }  
}
```

Contoh Pemrograman Python

```
class Mobil:
    def __init__(self, nama, warna, tahunPembuatan):
        self.nama = nama
        self.warna = warna
        self.__tahunPembuatan = tahunPembuatan
        self.__kecepatan = 0

    def ubahKecepatan(self, kecepatanBaru):
        self.__kecepatan = kecepatanBaru

    def __matikanMesin(self):
        self.__kecepatan = 0

    def informasiMobil(self):
        print("Nama mobil:", self.nama)
        print("Warna mobil:", self.warna)
        print("Tahun pembuatan:", self.__tahunPembuatan)
        print("Kecepatan saat ini:", self.__kecepatan)

mobil1 = Mobil("Avanza", "Hitam", 2015)

# Mengubah kecepatan mobil
mobil1.ubahKecepatan(60)
```

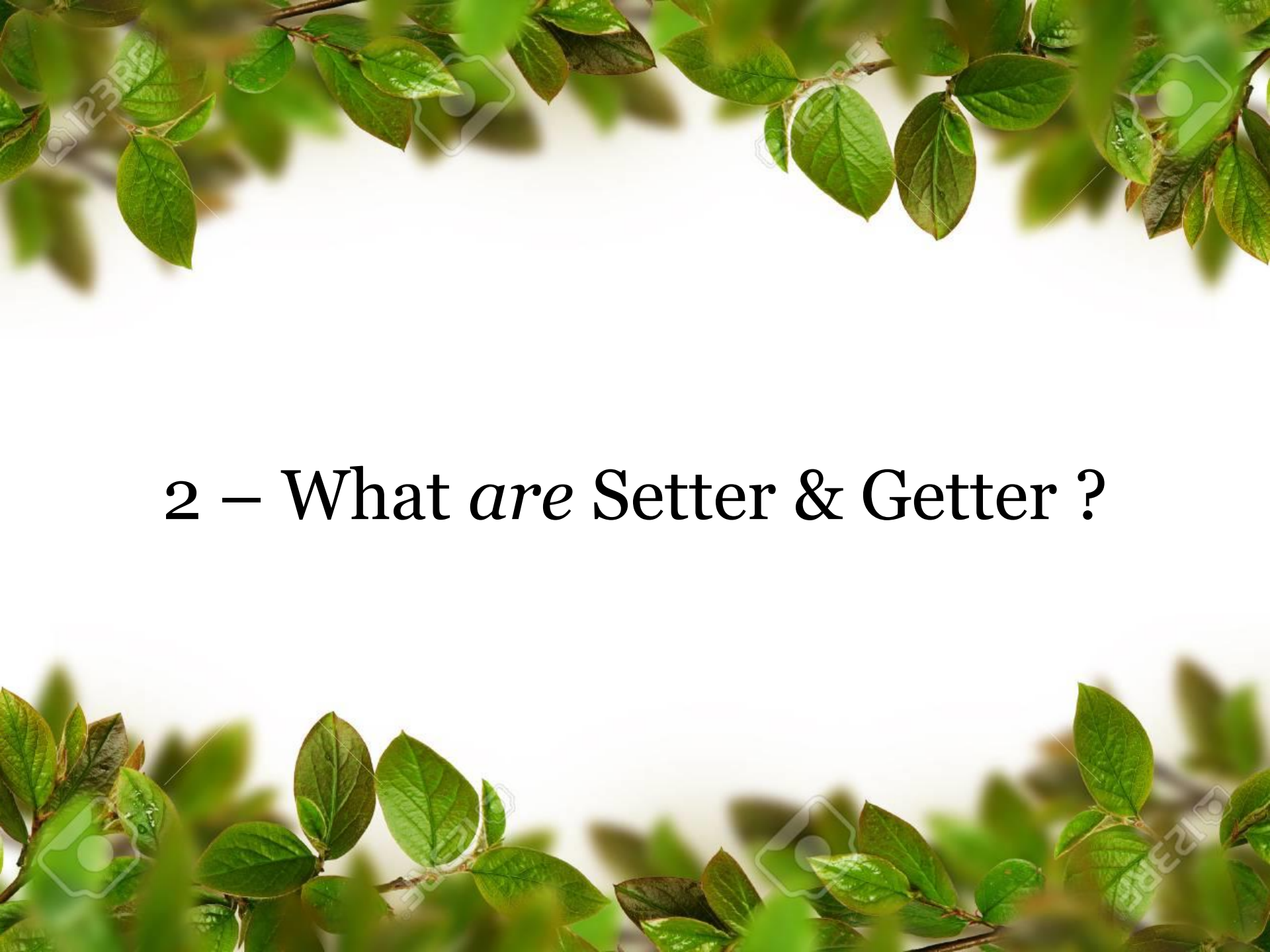
Atribut tahunPembuatan dan metode matikanMesin diatur sebagai private access dengan menambahkan double underscore pada awal nama atribut atau metode, seperti __tahunPembuatan dan __matikanMesin. Atribut dan metode tersebut hanya dapat diakses dari dalam objek Mobil, sehingga tidak dapat diakses langsung dari luar objek Mobil.

Untuk mengakses atribut dan metode private access dari luar objek Mobil, dapat menggunakan metode atau fungsi publik yang disediakan oleh objek Mobil, seperti ubahKecepatan dan informasiMobil. Sedangkan, jika mencoba untuk mengakses atribut atau metode private access secara langsung dari luar objek Mobil, seperti mobil1.__tahunPembuatan atau mobil1.__matikanMesin(), akan menghasilkan error.



Untuk menghindari adanya perubahan yang tidak diinginkan atau akses yang tidak sah dari luar objek terhadap variable/atribut, maka harus ada yang dikontrol dengan baik dan aman.

Ketika membahas tentang **member function** dan enkapsulasi (*hak akses public, private dan protected*), kadang kita akan bertemu dengan istilah **Setter** dan **Getter**.



2 – What *are* Setter & Getter ?

Setter & Getter Techniques

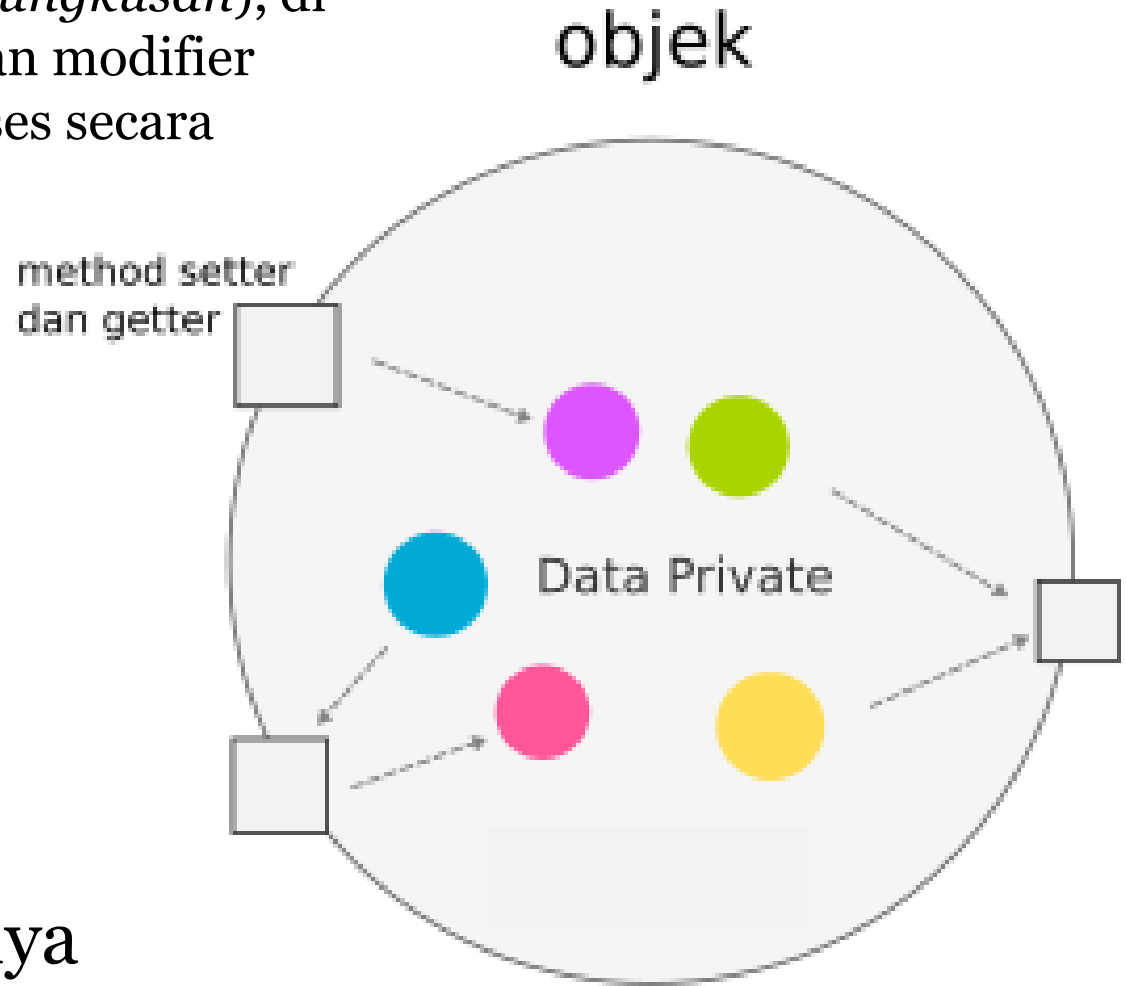
Setter = PENGATUR

Getter = PENGAMBIL

- ❖ Setter adalah *member method* yang dipakai untuk memberikan nilai ke dalam sebuah **data member (variable)**. Sedangkan Getter adalah *member method* yang dipakai untuk menampilkan nilai *data member*.
- ❖ Setter dan Getter adalah *metode* yang digunakan untuk mengatur dan mengakses nilai dari atribut atau properti suatu objek. Setter digunakan untuk mengatur atau mengubah nilai atribut, sedangkan Getter digunakan untuk mengakses nilai atribut dari luar class menggunakan *object*.

Dalam OOP, kita sering mendengar istilah encapsulation (*pembungkusan*), di mana data dibungkus dengan modifier `private` agar tidak bisa diakses secara langsung dari luar class.

Method setter dan getter adalah dua method yang tugasnya untuk mengambil dan mengisi data ke dalam objek.



Method *setter* dan *getter* yang akan membantu kita mengakses data tersebut.



Operator



Setter = PENGATUR

Seller



Getter = PENGAMBIL



Apakah perlu untuk mengubah semua data member suatu class dan di set sebagai **private** (bukan public) supaya aman...???

Setter & Getter Technique



Jika semua data member di set sebagai **private**, maka kita tidak bisa lagi memberikan nilai dan mengakses nilai data member secara langsung.

Why Setter & Getter?



- 1) To improve data security;
- 2) To make it easier to control attributes and methods;
- 3) We can make classes read-only and write-only;
- 4) Programmers can change part of the code without fear of impacting other code.

Why Setter & Getter?

- 1) Setters and getters **can help in implementing the concept of encapsulation or data wrapping**. In this way, we can ensure that data is only accessed and modified through the specified methods (*setters and getters*) thereby **protecting data from unwanted modifications and strengthening application security**.
- 2) Setters and getters **can help in controlling access to data within a class**. By setting **variables as private, data cannot be accessed directly from outside the class**. However, with setters and getters, we can control how and when data can be accessed and modified.
- 3) Setters **can be used to validate data before it is inserted into a variable**. Whereas, **Getter can be used to ensure that the data returned by the variable is correct**. This way, **we can ensure that the data received and returned by a class is valid and accurate**.
- 4) By using setters and getters, we **can easily modify how data is accessed and modified without changing the way data is used outside the class**. For example, if we want to change the type of data stored in a variable, we only need to change the getter and setter to reflect the change, but the way the data is used outside the class does not change.



Exercise *for* Students

Exercise #1

```
1 # create class
2 class Perpustakaan:
3     def __init__(self):
4         self.__nama = "";
5         self.__jumlah_buku = 0;
6
7     def set_nama(self, nama):
8         self.__nama = nama;
9
10    def get_nama(self):
11        return self.__nama;
12
13    def set_jumlah_buku(self, jumlah_buku):
14        self.__jumlah_buku = jumlah_buku;
15
16    def get_jumlah_buku(self):
17        return self.__jumlah_buku;
18
19 # Membuat objek perpustakaan dari class Perpustakaan
20 perpustakaan = Perpustakaan()
21
22 # Mengatur nilai atribut menggunakan setter
23 perpustakaan.set_nama("Perpustakaan Umum")
24 perpustakaan.set_jumlah_buku(10000)
25
26 # Mengakses nilai atribut menggunakan getter
27 nama_perpustakaan = perpustakaan.get_nama()
28 jumlah_buku = perpustakaan.get_jumlah_buku()
29 print("Nama perpustakaan:", nama_perpustakaan)
30 print("Jumlah buku di perpustakaan:", jumlah_buku)
31
32 # Mengubah nilai atribut menggunakan setter
33 perpustakaan.set_jumlah_buku(15000)
34
35 # Mengakses nilai atribut setelah diubah menggunakan getter
36 jumlah_buku = perpustakaan.get_jumlah_buku()
37 print("Jumlah buku di perpustakaan setelah diubah:", jumlah_buku)
```

Membuat objek perpustakaan dari class Perpustakaan, mengatur nilai atribut menggunakan setter, mengakses nilai atribut menggunakan getter, serta mengubah nilai atribut menggunakan setter dan mengakses nilai atribut setelah diubah menggunakan getter.

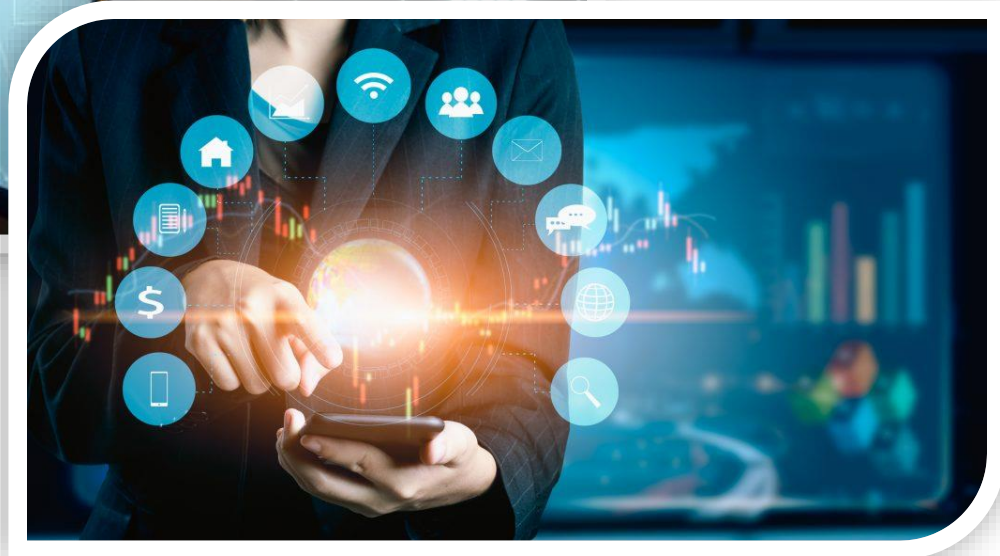
- 1) **Setter** tanpa *return value*.
- 2) **Getter** memiliki *return value*.

Exercise #2

```
1 # create class
2 class Perpustakaan:
3     def __init__(self):
4         self.__nama = "";
5         self.__jumlah_buku = 0;
6
7     def set_nama(self, nama):
8         if nama == "Unklab":
9             self.__nama = "Juan Manbu";
10        elif nama == "Klabat":
11            self.__nama = "Senny Taju";
12        else:
13            self.__nama = nama;
14
15    def get_nama(self):
16        if self.__nama == "Senny Taju":
17            return "He is a lecturer.";
18        else:
19            return "He/she is a student.";
20
21    def set_jumlah_buku(self, jumlah_buku):
22        self.__jumlah_buku = jumlah_buku;
23
24    def get_jumlah_buku(self):
25        return self.__jumlah_buku;
26
27
28 # Membuat objek perpus1 dari class Perpustakaan
29 perpus1 = Perpustakaan()
30
31 # Mengatur nilai atribut menggunakan setter
32 perpus1.set_nama("Klabat")
33 perpus1.set_jumlah_buku(10000)
34
35 # Mengakses nilai atribut menggunakan getter
36 nama_perpus = perpus1.get_nama()
37 jumlah_buku = perpus1.get_jumlah_buku()
38 print("Nama perpustakaan:", nama_perpus)
39 print("Jumlah buku di perpustakaan:", jumlah_buku)
40
41 # Mengubah nilai atribut menggunakan setter
42 perpus1.set_jumlah_buku(15000)
43
44 # Mengakses nilai atribut setelah diubah menggunakan getter
45 jumlah_buku = perpus1.get_jumlah_buku()
46 print("Jumlah buku di perpustakaan setelah diubah:", jumlah_buku)
47
```

Proses Validasi:

- 1) **Setter** dapat digunakan untuk memvalidasi data sebelum dimasukkan ke dalam variabel.
- 2) **Getter** dapat digunakan untuk memastikan bahwa data yang dikembalikan (*data yang keluar*) oleh variabel adalah benar.



A **banking information system** is a complex and sophisticated network of **computer hardware**, **software** and **databases** designed to manage and process financial data.

Banking Information System

Buatlah **sistem perbankan sederhana** menggunakan konsep/paradigma OOP dengan Python. Students dapat memulai dengan merancang fitur-fitur dasar yang umumnya ditemukan dalam sistem perbankan yaitu:

[1] Bank (Bank):

Informasi bank (nama, alamat, nomor telepon, dll.).
Manajemen cabang (jika bank memiliki lebih dari satu cabang).
Laporan keuangan (total aset, total kewajiban, laba/rugi, dll.).

[2] Account (Akun):

Pembuatan akun baru.
Informasi akun (nomor akun, saldo, pemilik, dll.).
Penghapusan akun.
Penyetoran (deposit).
Penarikan (withdrawal).
Transfer antar akun.

[3] Customer (Nasabah):

Pendaftaran nasabah baru.
Informasi nasabah (nama, alamat, nomor telepon, dll.).
Menghapus nasabah.

[4] Transaction (Transaksi):

Riwayat transaksi.
Detail transaksi (tanggal, jenis transaksi, jumlah, akun terkait, dll.).

[5] Authentication (Autentikasi):

Sistem login untuk karyawan bank.
Verifikasi identitas untuk nasabah.

[6] Interest (Bunga):

Perhitungan bunga pada saldo akun.

[7] Loan (Pinjaman):

Pemberian pinjaman.
Pelacakan pembayaran pinjaman.

[8] Security (Keamanan):

Perlindungan data nasabah.
Sistem keamanan untuk transaksi.

[9] Notification (Notifikasi):

Pemberitahuan kepada nasabah tentang transaksi atau informasi akun.

[10] Customer Support (Dukungan Pelanggan):

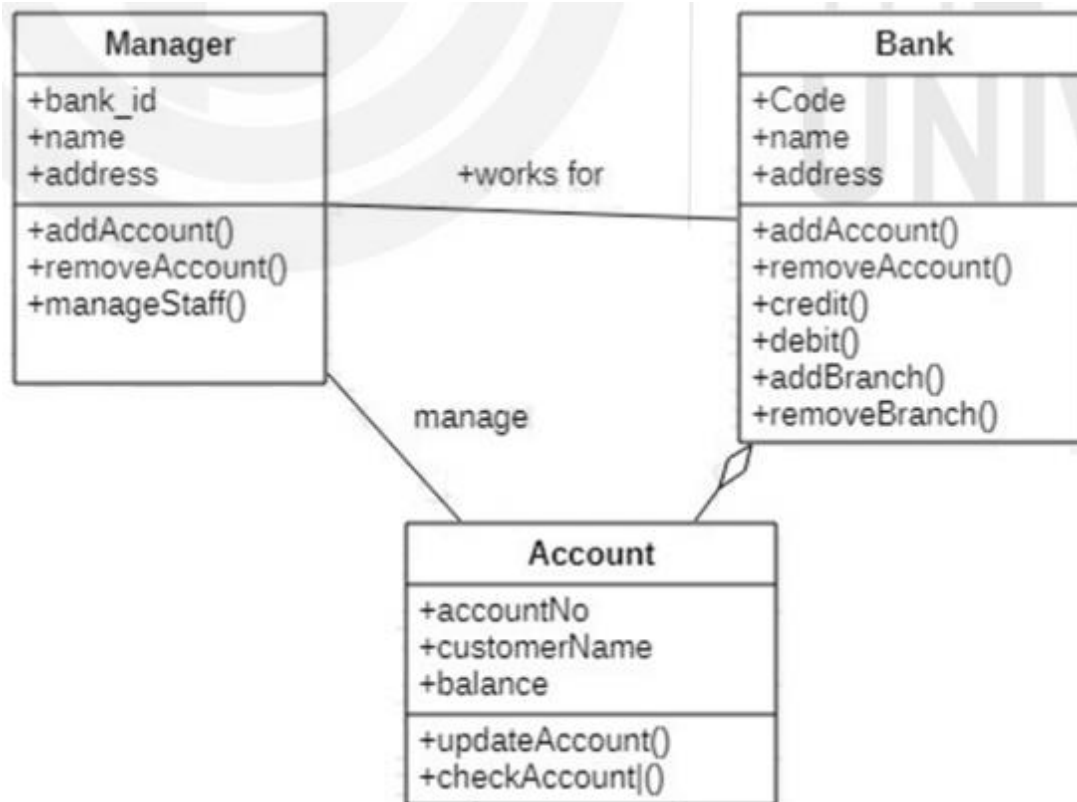
Layanan bantuan untuk nasabah.
Sistem tiket untuk masalah atau pertanyaan pelanggan.

[11] Currency Exchange (Penukaran Mata Uang):

Layanan untuk menukar mata uang.

[12] ATM (Automatic Teller Machine):

Pengelolaan transaksi melalui mesin ATM.



Menurut anda, apakah tipe hubungan diatas menggunakan konsep pewarisan??? Atau ada cara lain untuk menghubungkan dua atau lebih class diagram?

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

